ENV 797 - Time Series Analysis for Energy and Environment Applications | Spring 2026

Assignment 5 - Due date 02/17/26

Mingjie Wei

Directions

You should open the .rmd file corresponding to this assignment on RStudio. The file is available on our class repository on Github.

Once you have the file open on your local machine the first thing you will do is rename the file such that it includes your first and last name (e.g., "LuanaLima_TSA_A05_Sp26.Rmd"). Then change "Student Name" on line 3 with your name.

Then you will start working through the assignment by **creating code and output** that answer each question. Be sure to use this assignment document. Your report should contain the answer to each question and any plots/tables you obtained (when applicable).

When you have completed the assignment, **Knit** the text and code into a single PDF file. Submit this pdf using Canvas.

R packages needed for this assignment: "readxl", "ggplot2", "forecast","tseries", and "Kendall". Install these packages, if you haven't done yet. Do not forget to load them before running your script, since they are NOT default packages.\

```
#Load/install required package here
library(forecast)
library(tseries)
library(ggplot2)
library(Kendall)
library(lubridate)
library(tidyverse)  #load this package so yon clean the data frame using pipes
library(readxl)
```

Consider the same data you used for A04 from the spreadsheet "Table_10.1_Renewable_Energy_Production_and_Consumptio
The data comes from the US Energy Information and Administration and corresponds to the December 2025 Monthly Energy Review.

```
#Importing data set - using readxl package
energy_data <- read_excel(
  path="../Data/Table_10.1_Renewable_Energy_Production_and_Consumption_by_Source.xlsx",
  skip = 12,
  sheet="Monthly Data",
  col_names=FALSE
  )
```

```
## New names:
## * `` -> `...1`
## * `` -> `...2`
```

```
## * `` -> `...3`
## * `` -> `...4`
## * `` -> `...5`
## * `` -> `...6`
## * `` -> `...7`
## * `` -> `...8`
## * `` -> `...9`
## * `` -> `...10`
## * `` -> `...11`
## * `` -> `...12`
## * `` -> `...13`
## * `` -> `...14`
```

```r
#Now let's extract the column names from row 11 only
read_col_names <- read_excel(
  path="../Data/Table_10.1_Renewable_Energy_Production_and_Consumption_by_Source.xlsx",
  skip = 10,
  n_max = 1,
  sheet="Monthly Data",
  col_names=FALSE
  )
```

```
## New names:
## * `` -> `...1`
## * `` -> `...2`
## * `` -> `...3`
## * `` -> `...4`
## * `` -> `...5`
## * `` -> `...6`
## * `` -> `...7`
## * `` -> `...8`
## * `` -> `...9`
## * `` -> `...10`
## * `` -> `...11`
## * `` -> `...12`
## * `` -> `...13`
## * `` -> `...14`
```

```r
colnames(energy_data) <- read_col_names
nobs <- nrow(energy_data)

nobs=nrow(energy_data)
nvar=ncol(energy_data)

head(energy_data)
```

```
## # A tibble: 6 x 14
##   Month               `Wood Energy Production` `Biofuels Production`
##   <dttm>                                  <dbl> <chr>
## 1 1973-01-01 00:00:00                      130. Not Available
## 2 1973-02-01 00:00:00                      117. Not Available
## 3 1973-03-01 00:00:00                      130. Not Available
## 4 1973-04-01 00:00:00                      125. Not Available
## 5 1973-05-01 00:00:00                      130. Not Available
## 6 1973-06-01 00:00:00                      125. Not Available
## # i 11 more variables: `Total Biomass Energy Production` <dbl>,
```

```
## #   `Total Renewable Energy Production` <dbl>,
## #   `Hydroelectric Power Consumption` <dbl>,
## #   `Geothermal Energy Consumption` <dbl>, `Solar Energy Consumption` <chr>,
## #   `Wind Energy Consumption` <chr>, `Wood Energy Consumption` <dbl>,
## #   `Waste Energy Consumption` <dbl>, `Biofuels Consumption` <chr>,
## #   `Total Biomass Energy Consumption` <dbl>, ...
```

Handling Missing Data

Q1

Using the original dataset, create a new data frame that includes only the following variables: **Date, Solar Energy Consumption and Wind Energy Consumption**. Check the class of columns, you will see that they are stored are characters instead of numbers. Because solar generation begins later in the sample, the early observations are recorded as "Not Available". Convert the data to numeric, the "Not Available" will became NAs.

You may either filter out the "Not Available" rows and then convert the column to numeric or convert first and then remove missing values using drop_na() (or na.omit()). If you are comfortable using pipes for data wrangling, please do so.

Important: Note that we dropping the missing observations instead of interpolating is becasue they only happen in the beginning of the series!

```r
energy_sw <- energy_data %>%
  transmute(
    Date = as.Date(Month),
    Solar = as.numeric(`Solar Energy Consumption`),
    Wind  = as.numeric(`Wind Energy Consumption`)
  ) %>%
  drop_na()
```

```
## Warning: There were 2 warnings in `transmute()`.
## The first warning was:
## i In argument: `Solar = as.numeric(`Solar Energy Consumption`)`.
## Caused by warning:
## ! NAs introduced by coercion
## i Run `dplyr::last_dplyr_warnings()` to see the 1 remaining warning.
```

```r
glimpse(energy_sw)
```

```
## Rows: 501
## Columns: 3
## $ Date  <date> 1984-01-01, 1984-02-01, 1984-03-01, 1984-04-01, 1984-05-01, 198~
## $ Solar <dbl> 0.000, 0.000, 0.001, 0.001, 0.002, 0.003, 0.001, 0.003, 0.003, 0~
## $ Wind  <dbl> 0.000, 0.001, 0.001, 0.002, 0.003, 0.002, 0.002, 0.001, 0.002, 0~
```

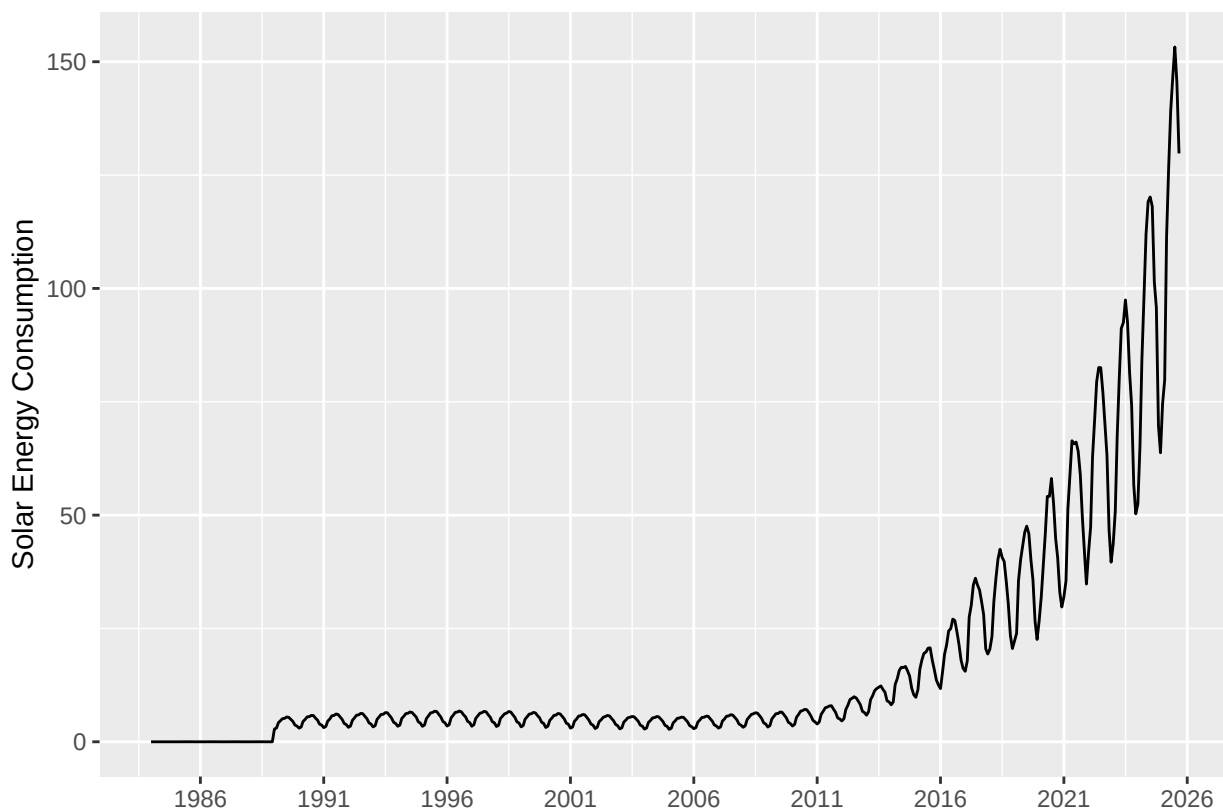
```r
summary(energy_sw)
```

```
##       Date                Solar              Wind
##  Min.   :1984-01-01   Min.   :  0.000   Min.   :  0.000
##  1st Qu.:1994-06-01   1st Qu.:  3.924   1st Qu.:  0.844
##  Median :2004-11-01   Median :  5.658   Median :  3.999
##  Mean   :2004-10-31   Mean   : 16.623   Mean   : 31.305
##  3rd Qu.:2015-04-01   3rd Qu.: 15.758   3rd Qu.: 55.001
##  Max.   :2025-09-01   Max.   :153.256   Max.   :172.670
```

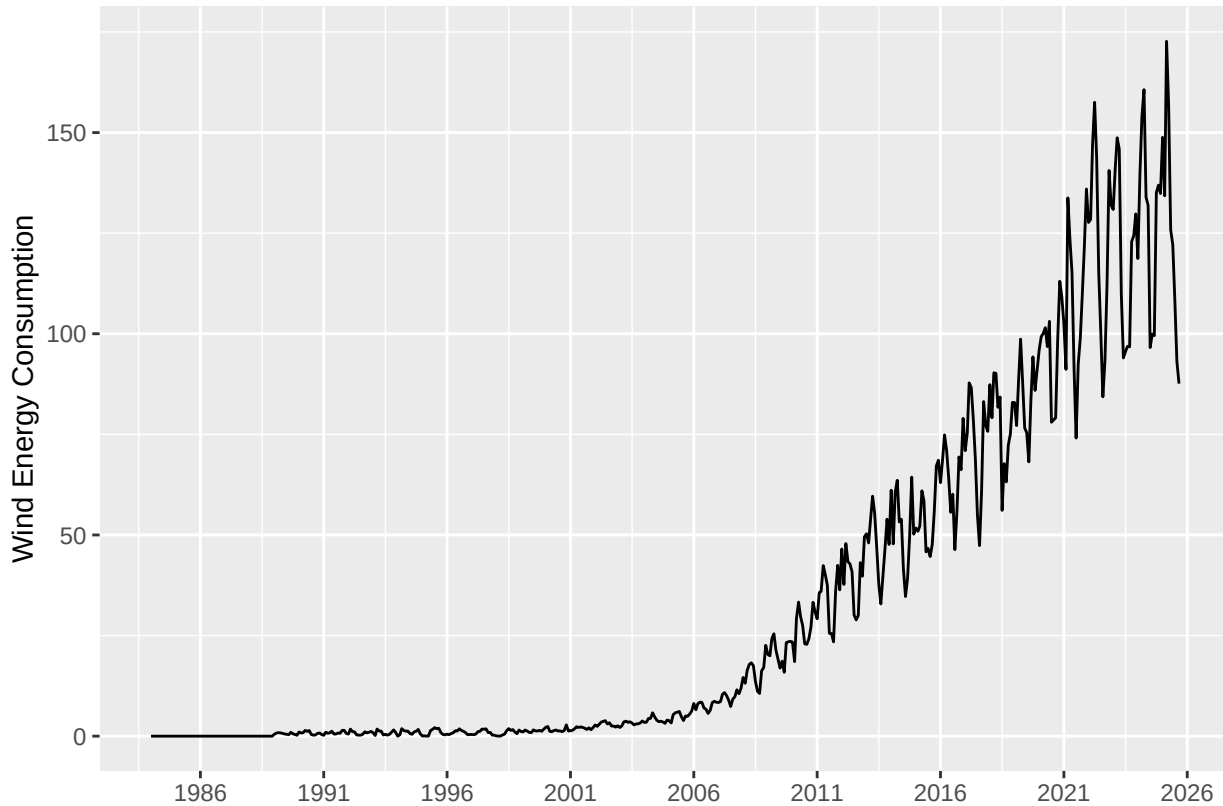
Q2

Plot the Solar and Wind energy consumption over time using ggplot. Plot each series on a separate graph. No need to add legend. Add informative names to the y axis using `ylab()`. Explore the function `scale_x_date()` on ggplot and see if you can change the x axis to improve your plot. Hint: use `scale_x_date(date_breaks = "5 years", date_labels = "%Y")")`

```
p_solar <- ggplot(energy_sw, aes(x = Date, y = Solar)) +
  geom_line() +
  ylab("Solar Energy Consumption") +
  xlab("") +
  scale_x_date(date_breaks = "5 years", date_labels = "%Y")

p_wind <- ggplot(energy_sw, aes(x = Date, y = Wind)) +
  geom_line() +
  ylab("Wind Energy Consumption") +
  xlab("") +
  scale_x_date(date_breaks = "5 years", date_labels = "%Y")

p_solar
```



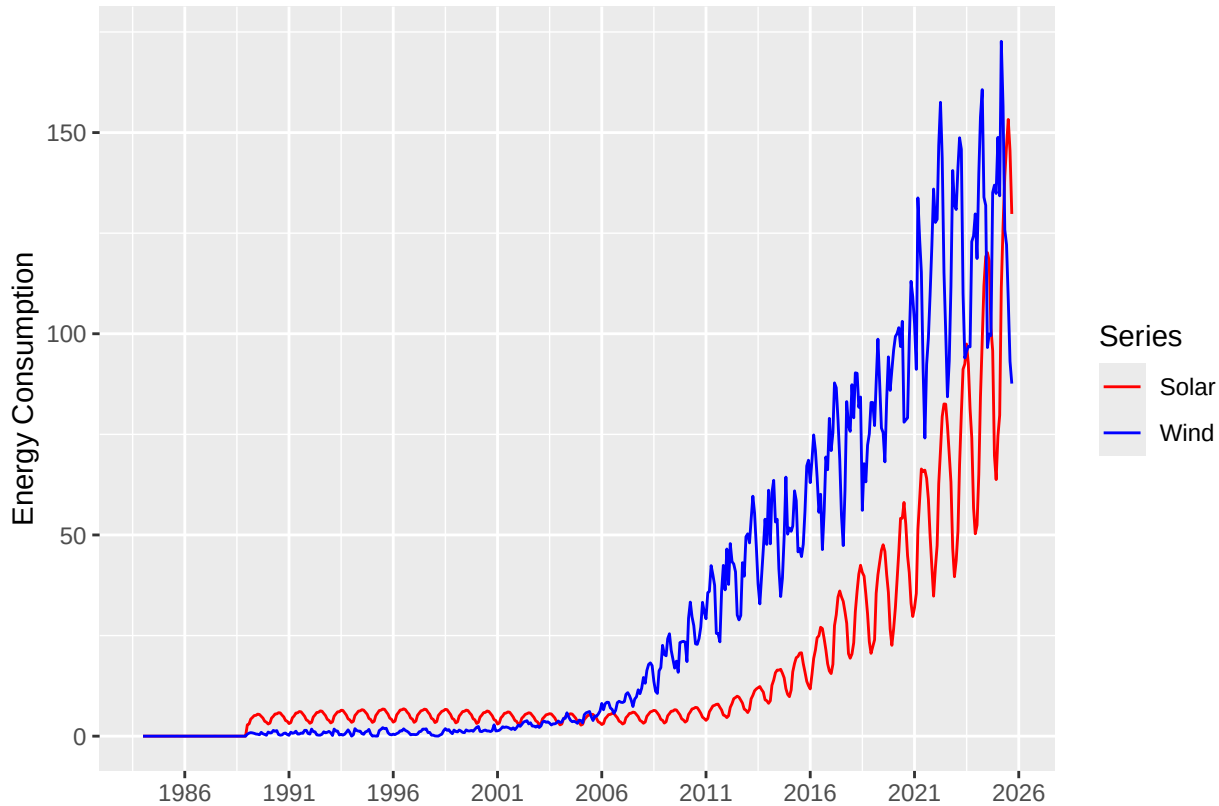

```
p_wind
```



Q3

Now plot both series in the same graph, also using ggplot(). Use function `scale_color_manual()` to manually add a legend to ggplot. Make the solar energy consumption red and wind energy consumption blue. Add informative name to the y axis using `ylab("Energy Consumption)`. And use function `scale_x_date()` to set x axis breaks every 5 years.

```
energy_long <- energy_sw %>%
  pivot_longer(cols = c(Solar, Wind), names_to = "Series", values_to = "Value")

p_both <- ggplot(energy_long, aes(x = Date, y = Value, color = Series)) +
  geom_line() +
  ylab("Energy Consumption") +
  xlab("") +
  scale_x_date(date_breaks = "5 years", date_labels = "%Y") +
  scale_color_manual(values = c(Solar = "red", Wind = "blue"))

p_both
```



Decomposing the time series

The stats package has a function called decompose(). This function only take time series object. As the name says the decompose function will decompose your time series into three components: trend, seasonal and random. This is similar to what we did in the previous script, but in a more automated way. The random component is the time series without seasonal and trend component.

Additional info on `decompose()`.

1) You have two options: alternative and multiplicative. Multiplicative models exhibit a change in frequency over time.
2) The trend is not a straight line because it uses a moving average method to detect trend.
3) The seasonal component of the time series is found by subtracting the trend component from the original data then grouping the results by month and averaging them.
4) The random component, also referred to as the noise component, is composed of all the leftover signal which is not explained by the combination of the trend and seasonal component.

Q4

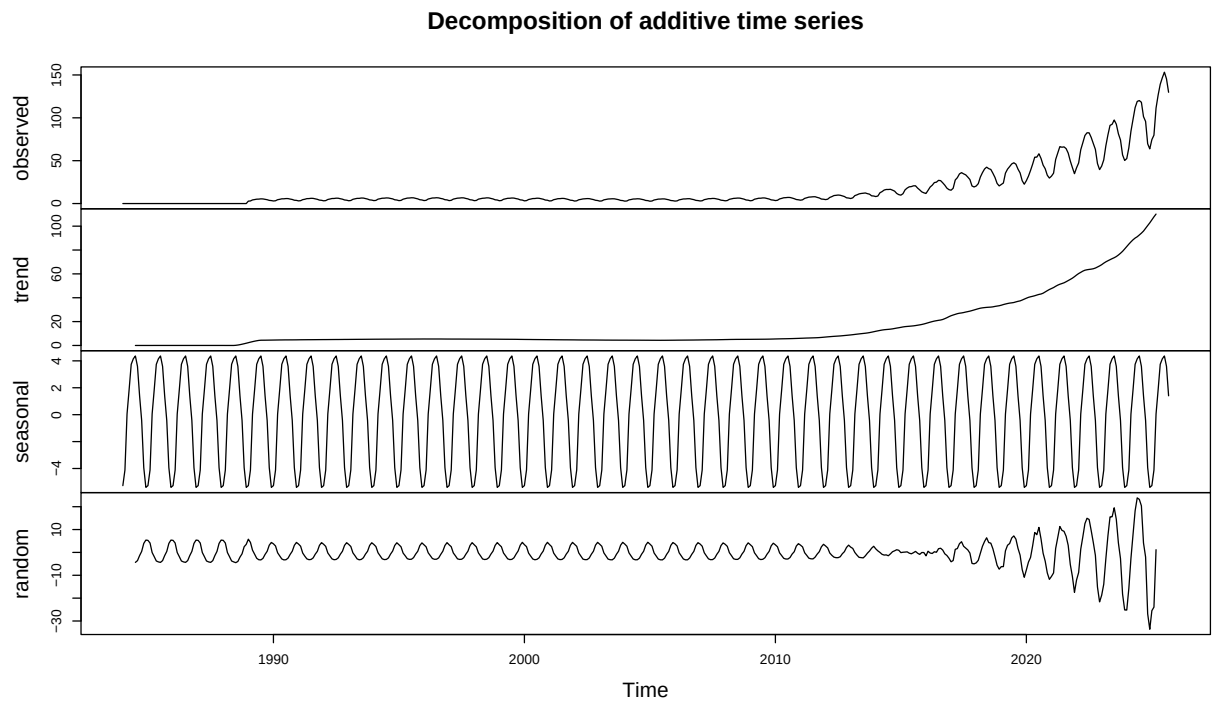
Transform wind and solar series into a time series object and apply the decompose function on them using the additive option, i.e., `decompose(ts_data, type = "additive")`. What can you say about the trend component? What about the random component? Does the random component look random? Or does it appear to still have some seasonality on it?

```
start_year  <- year(min(energy_sw$Date))
start_month <- month(min(energy_sw$Date))

ts_solar <- ts(energy_sw$Solar, start = c(start_year, start_month), frequency = 12)
```

```r
ts_wind  <- ts(energy_sw$Wind,  start = c(start_year, start_month), frequency = 12)

dec_solar_add <- decompose(ts_solar, type = "additive")
dec_wind_add  <- decompose(ts_wind,  type = "additive")
```

```r
plot(dec_solar_add)
```

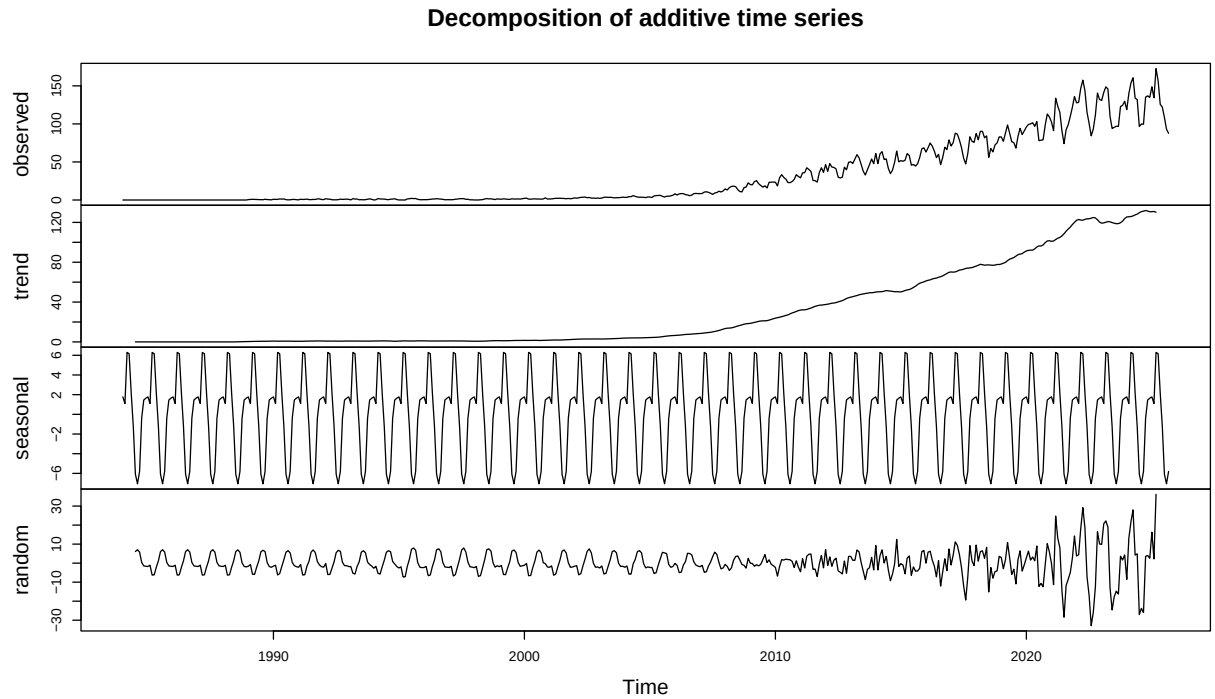
Solar - Additive Decomposition



- Trend: The trend is relatively flat for many years and then rises sharply after around the 2010s, indicating rapid long-term growth in solar consumption.
- Random: The remainder is not purely random, especially in recent years where the variability becomes much larger, suggesting increasing fluctuations and that an additive model may not fully capture changing seasonal strength at higher levels.

```r
plot(dec_wind_add)
```

Wind - Additive Decomposition



- Trend: The trend shows a clear long-term increase, accelerating after around the late 2000s/early 2010s, reflecting sustained growth in wind consumption.
- Random: The remainder becomes much more volatile in recent years and contains large spikes, so it is not white-noise-like; this indicates stronger irregular shocks/outliers and possible level-dependent variation not well handled by a simple additive decomposition.

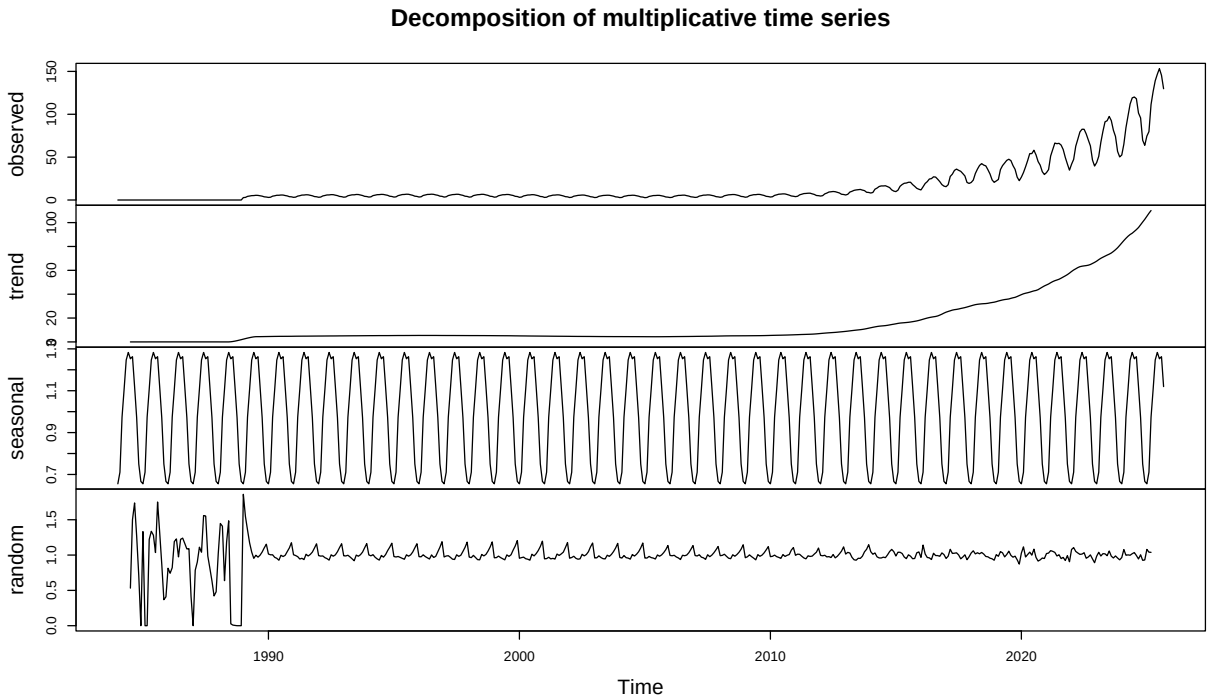
Q5

Use the decompose function again but now change the type of the seasonal component from additive to multiplicative. What happened to the random component this time?

```
dec_solar_mul <- decompose(ts_solar, type = "multiplicative")
dec_wind_mul  <- decompose(ts_wind,  type = "multiplicative")
```

```
plot(dec_solar_mul)
```

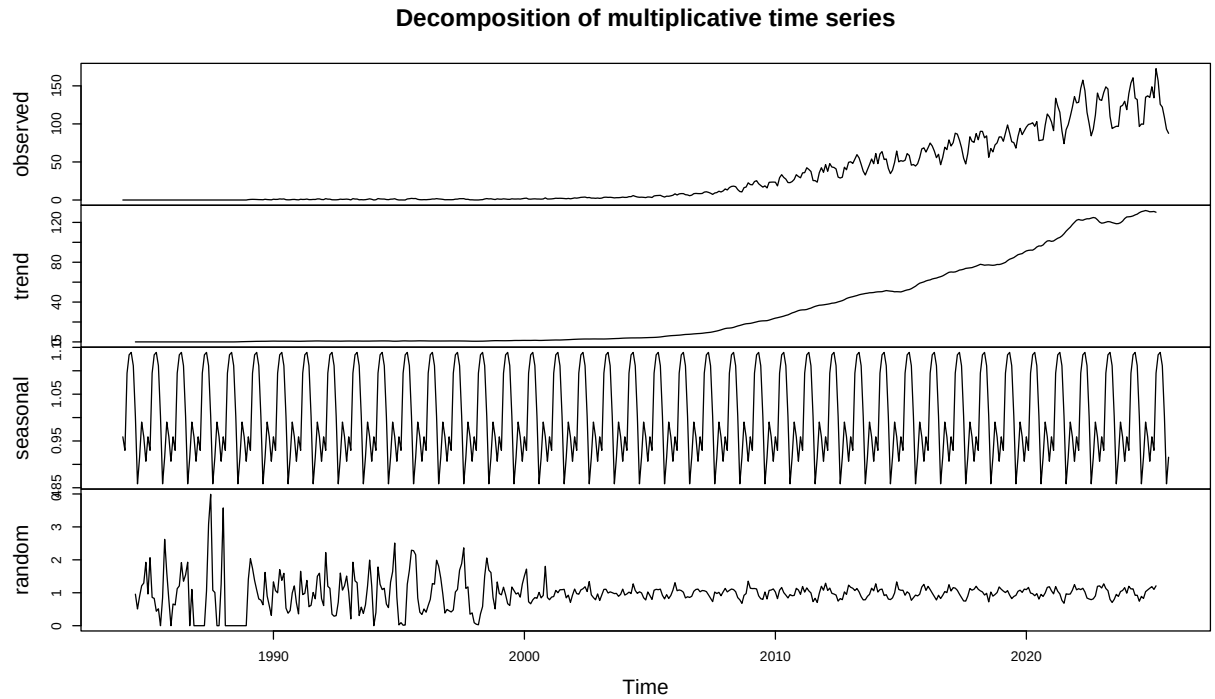
Solar - Multiplicative Decomposition



- Random: The remainder is much more stable (closer to white noise) than in the additive case, especially in the later period, suggesting the seasonal/irregular variation scales with the series level.

```
plot(dec_wind_mul)
```

Wind - Multiplicative Decomposition



- Random: The remainder becomes more uniform over time compared with the additive decomposition, though there are still some irregular spikes early in the sample; overall it looks closer to random noise when seasonality is modeled multiplicatively.

Q6

When fitting a model to this data, do you think you need all the historical data? Think about the data from 80s, 90s and early 20s. Are there any information from those years we might need to forecast the next six months of Solar and/or Wind consumption. Explain your response.

> Answer:
>
> For forecasting the next six months, we do not necessarily need the entire historical record from the 1980s–early 2000s. Short-horizon forecasts are driven mostly by the most recent level, trend, and seasonal pattern, which are better represented by the last several years. The older periods were at a much lower consumption level and may reflect different technology, capacity, and policy regimes, so including them could bias the estimated trend.
>
> However, older data can still be useful if we want a more stable estimate of monthly seasonality (assuming seasonality is consistent over time) or to understand long-run structural changes. Overall, a recent window (e.g., last 10–15 years) is typically more relevant for a 6-month forecast than using all historical data.

Q7

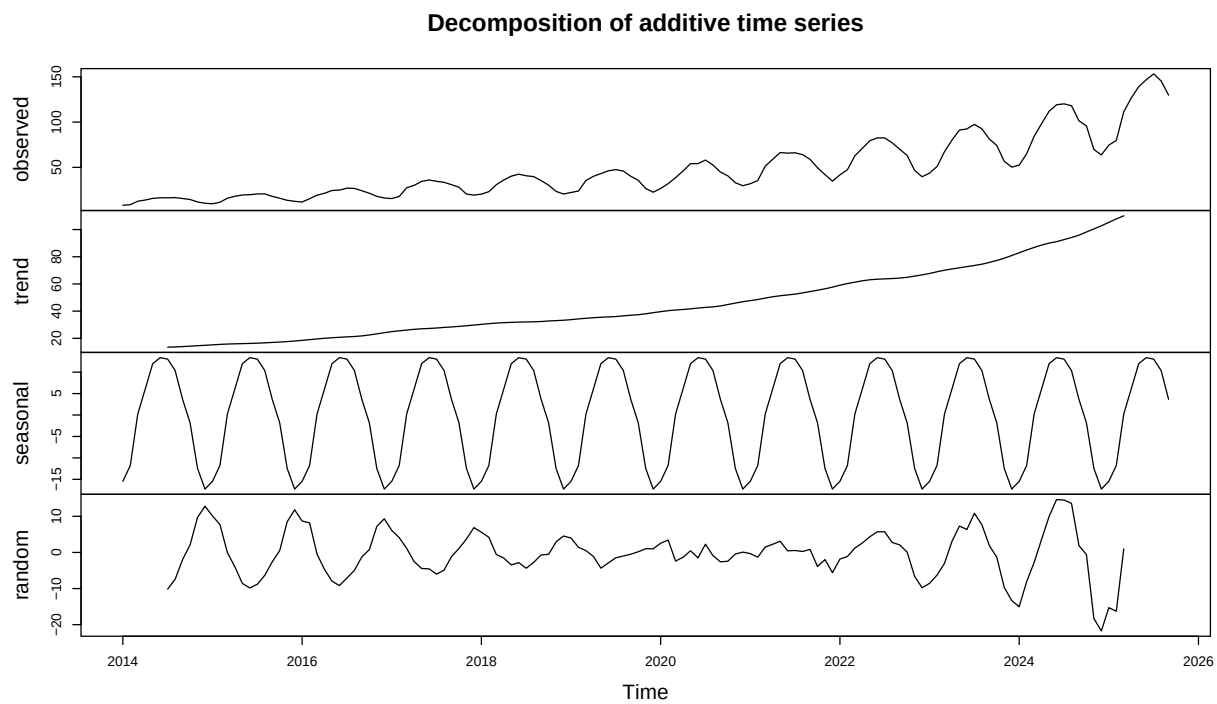
Create a new time series object where historical data starts on January 2014. Hint: use `filter()` function so that you don't need to point to row numbers, .i.e, `filter(xxxx, year(Date) >= 2014 )`. Apply the decompose function `type=additive` to this new time series. Comment on the results. Does the random component look random?

```
energy_sw_2014 <- energy_sw %>%
  filter(year(Date) >= 2014)
```

```r
ts_solar_2014 <- ts(energy_sw_2014$Solar,
                    start = c(year(min(energy_sw_2014$Date)), month(min(energy_sw_2014$Date))),
                    frequency = 12)

ts_wind_2014 <- ts(energy_sw_2014$Wind,
                   start = c(year(min(energy_sw_2014$Date)), month(min(energy_sw_2014$Date))),
                   frequency = 12)

dec_solar_2014_add <- decompose(ts_solar_2014, type = "additive")
dec_wind_2014_add  <- decompose(ts_wind_2014,  type = "additive")
```

```r
plot(dec_solar_2014_add)
```

Solar - Additive Decomposition - 2014+

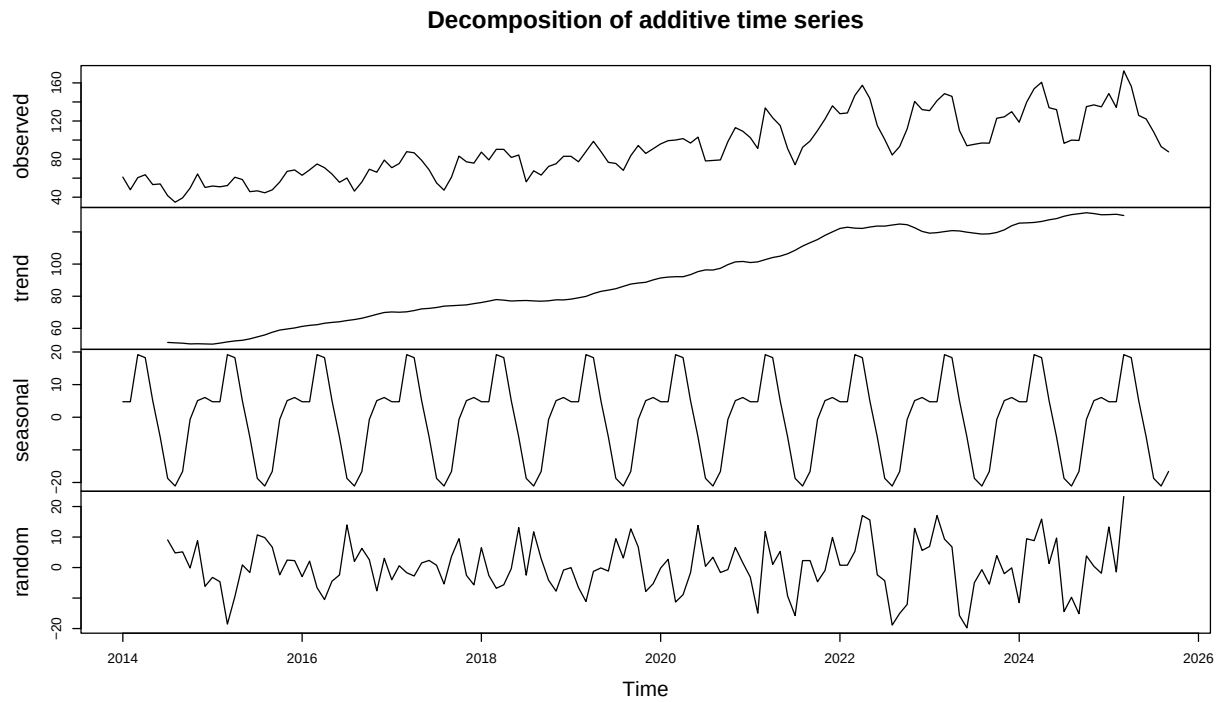


Answer:

- Trend: The trend shows a clear upward pattern from 2014 onward, with faster growth in the later years, indicating strong long-term expansion in solar consumption.
- Random: The remainder is not fully random. It still shows noticeable low-frequency swings and larger deviations in the most recent years, suggesting remaining structure and possible changing variance that is not fully captured by an additive decomposition.

```r
plot(dec_wind_2014_add)
```

Wind - Additive Decomposition - 2014+



Answer:

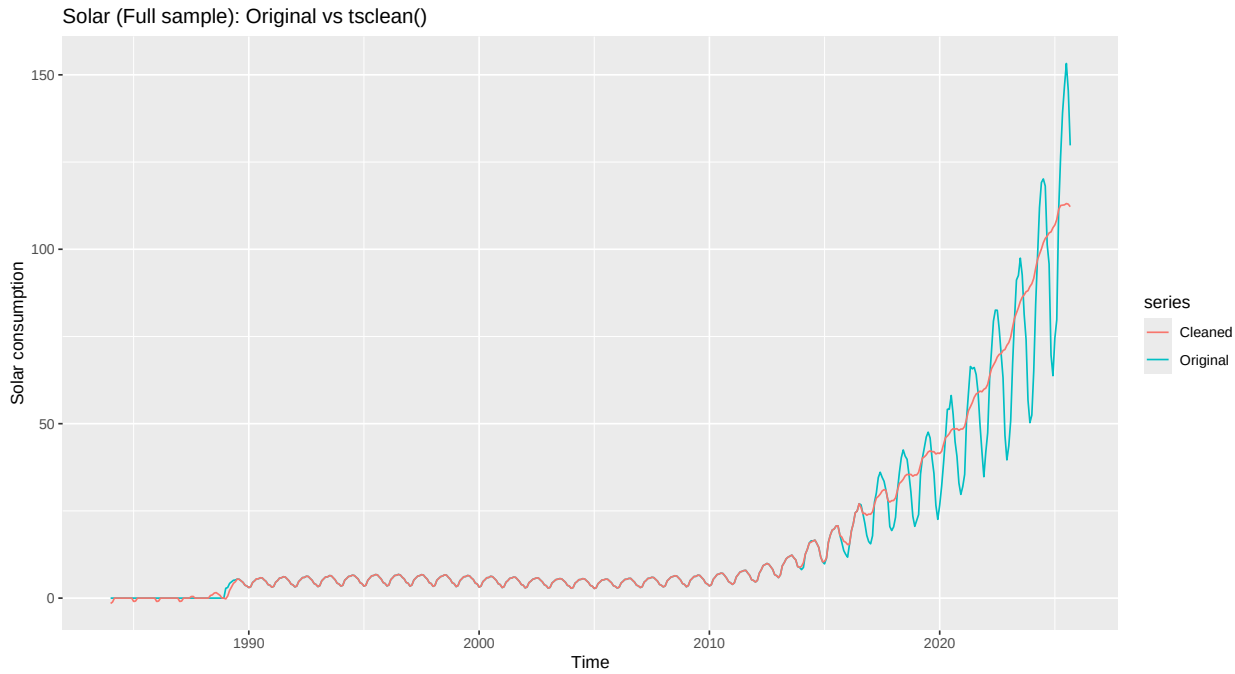
- Trend: The trend increases steadily from 2014, with growth slowing/flattening somewhat in the most recent years, suggesting wind consumption is still rising but at a more stable pace recently.
- Random: The remainder looks more noise-like than the full-sample additive decomposition, but it is not perfectly random. There are clustered fluctuations and a few larger shocks, indicating some irregular variation remains after removing trend and seasonality.

Identify and Remove outliers

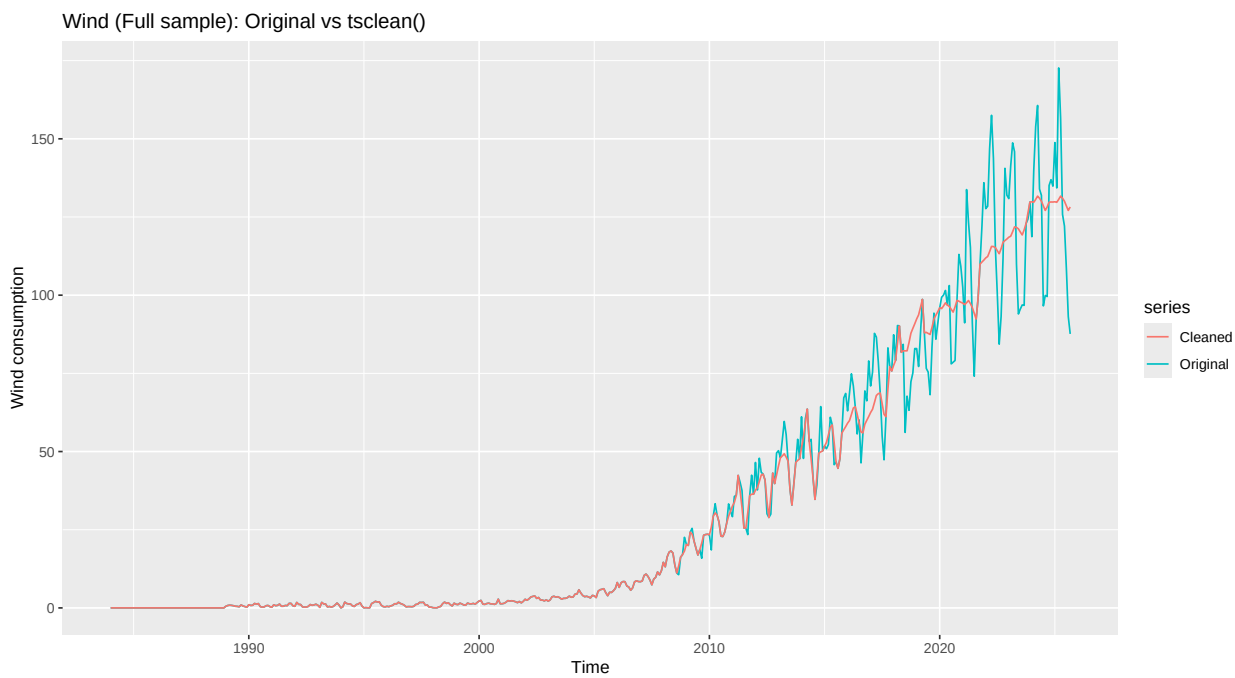
Q8

Apply the `tsclean()` to both time series object you created on Q4. Did the function removed any outliers from the series? Hint: Use `autoplot()` to check if there is difference between cleaned series and original series.

```
solar_clean <- tsclean(ts_solar)
wind_clean  <- tsclean(ts_wind)

# Solar comparison
autoplot(ts_solar, series = "Original") +
  autolayer(solar_clean, series = "Cleaned") +
  ggtitle("Solar (Full sample): Original vs tsclean()") +
  ylab("Solar consumption") + xlab("Time")
```




```r
# Wind comparison
autoplot(ts_wind, series = "Original") +
  autolayer(wind_clean, series = "Cleaned") +
  ggtitle("Wind (Full sample): Original vs tsclean()") +
  ylab("Wind consumption") + xlab("Time")
```



Answer:

- Solar: The cleaned series differs from the original, especially in the most recent years where extreme peaks are reduced. This indicates that `tsclean()` detected and replaced some

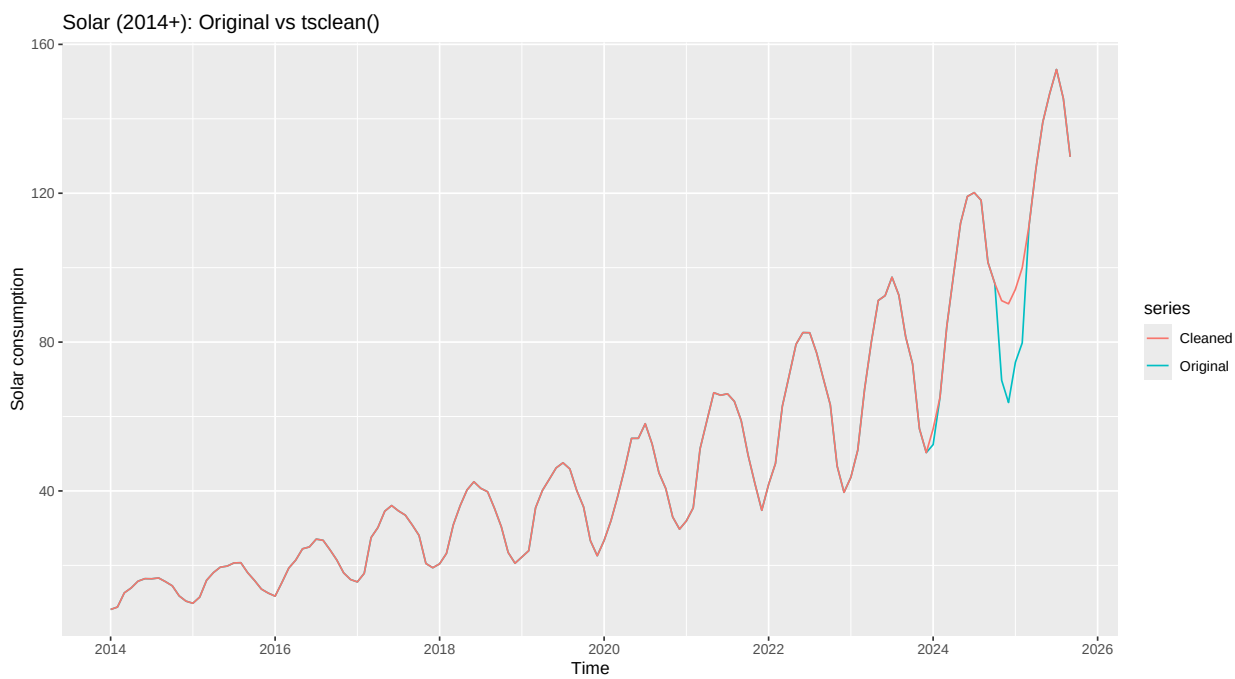
outliers/extreme observations in the full sample.

- Wind: The cleaned series also shows visible differences from the original, mainly by smoothing several large spikes in the later years. Therefore, `tsclean()` removed (replaced) some outliers in the full-sample wind series.

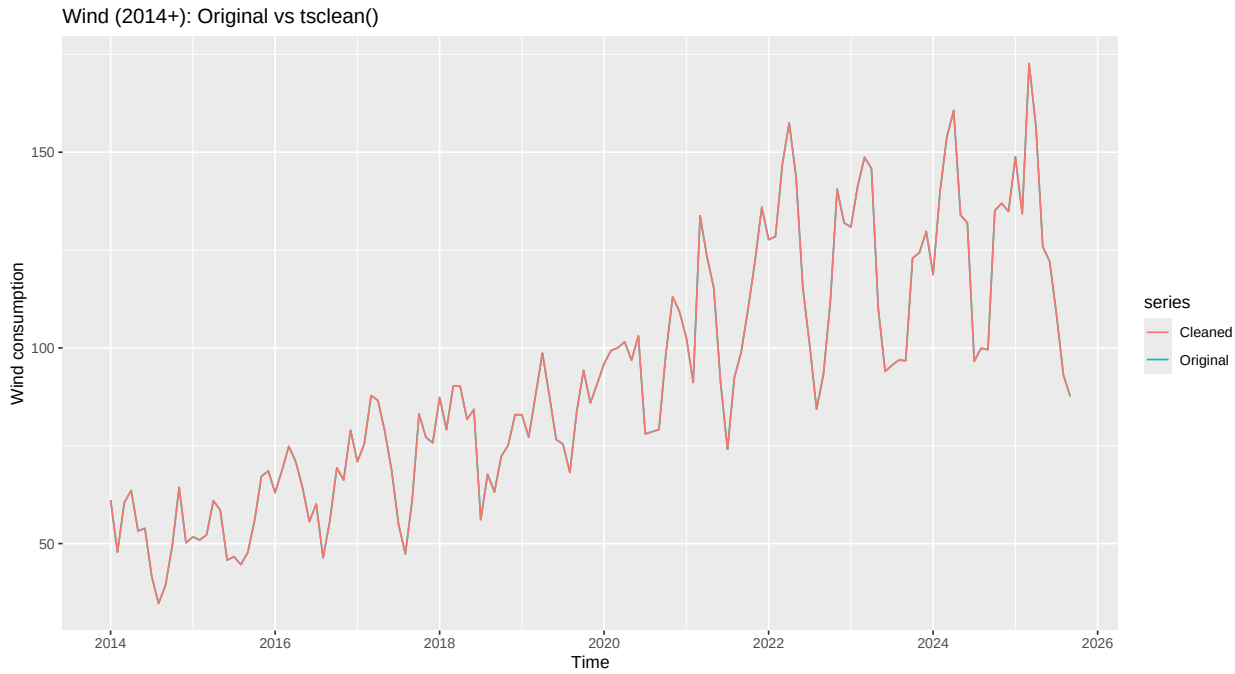
Q9

Redo number Q8 but now with the time series you created on Q7, i.e., the series starting in 2014. Using what `autoplot()` again what happened now? Did the function removed any outliers from the series?

```
solar_2014_clean <- tsclean(ts_solar_2014)
wind_2014_clean  <- tsclean(ts_wind_2014)

# Solar (2014+) comparison
autoplot(ts_solar_2014, series = "Original") +
  autolayer(solar_2014_clean, series = "Cleaned") +
  ggtitle("Solar (2014+): Original vs tsclean()") +
  ylab("Solar consumption") + xlab("Time")
```




```
# Wind (2014+) comparison
autoplot(ts_wind_2014, series = "Original") +
  autolayer(wind_2014_clean, series = "Cleaned") +
  ggtitle("Wind (2014+): Original vs tsclean()") +
  ylab("Wind consumption") + xlab("Time")
```




```r
all.equal(as.numeric(wind_2014_clean), as.numeric(ts_wind_2014))
```

```
## [1] TRUE
```

```r
max(abs(wind_2014_clean - ts_wind_2014))
```

```
## [1] 0
```

```r
sum(abs(wind_2014_clean - ts_wind_2014) > 1e-8)
```

```
## [1] 0
```

Answer:

- Solar (2014+): The cleaned series is very close to the original, with only minor adjustments near the end of the sample, suggesting few outliers in this period.
- Wind (2014+): The cleaned series is identical to the original series (no changes at all). This means `tsclean()` did not identify any outliers in the 2014+ wind series; the observed variability is treated as normal variation rather than outliers.